

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:25

ANNEXURE A APPLICATION BASED QUESTIONS

Code of Conduct:

I CH. Gokul Ram Sai (192424229) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: CH. Gokul Ram Sai

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure

accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution

Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand
---------	----	--	--	---------------------------------	--

APPLICATION BASED QUESTIONS

Question 1

Problem Statement

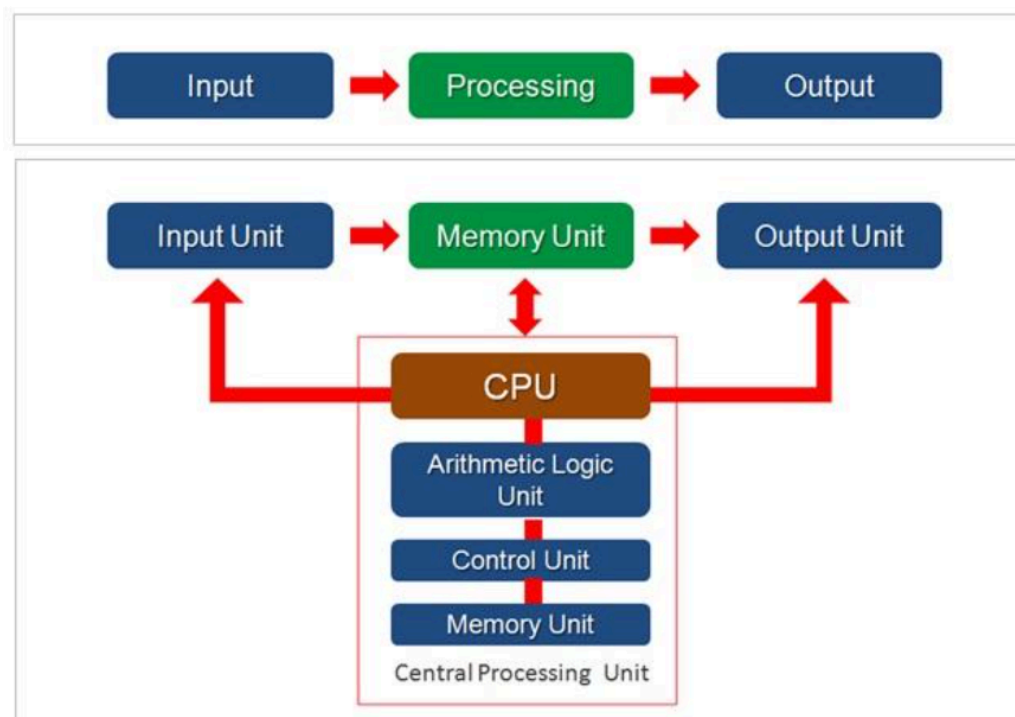
A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in the instruction execution cycle can enhance system performance.

1. Understanding the Problem Context

A smart city surveillance system uses multiple CCTV cameras to monitor roads, public places, and buildings. These cameras continuously capture high-resolution video and send it to the computer system for processing. The CPU analyzes each video frame to detect suspicious activities such as theft, accidents, or unauthorized access.

During peak hours, many cameras send video simultaneously. This increases the amount of data transferred between the **Input/Output (I/O) devices, Memory (RAM), and CPU**. As a result, the CPU becomes overloaded, memory access becomes slower, and the system bus experiences congestion. These issues increase response time and delay threat detection.

Figure 1: Functional Units of a Computer



Caption: Figure 1. Data flow in a smart surveillance system.

2. Application of Concepts

Interaction Between CPU, Memory, and I/O

CPU

The CPU is responsible for:

- Fetching video frames from memory.
- Executing image-processing algorithms.
- Detecting threats.
- Sending results to the monitoring system.

If too many frames arrive simultaneously, the CPU cannot process them quickly, increasing execution time.

Memory (RAM)

Memory stores incoming video frames temporarily.

Functions:

- Stores camera data.
- Supplies instructions to CPU.
- Stores intermediate processing results.

When RAM becomes full or memory bandwidth is low, the CPU waits for data, reducing performance.

Input/Output Devices

I/O devices include:

- CCTV Cameras
- Network Interfaces
- Storage Devices

They continuously transfer video data into memory.

If the transfer rate is slower than incoming data, queues are formed, creating delays.

Table 1: Functions of CPU, Memory, and I/O

Component	Function	Problem During Peak Hours
CPU	Processes video frames	High CPU utilization
Memory	Stores temporary data	Memory bottleneck
I/O Devices	Receive camera feeds	Slow transfer speed

3. Bus Structures

Single Bus Architecture

In a single-bus architecture:

- CPU
- Memory
- I/O Devices

share the same communication bus.

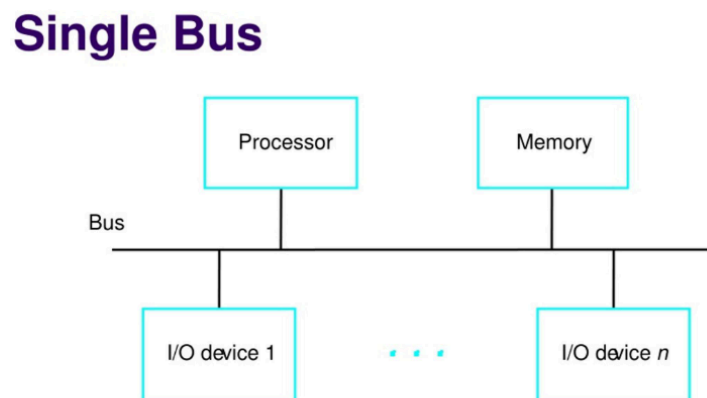
Advantages

- Low cost
- Simple design

Disadvantages

- Bus congestion
- Only one transfer at a time
- CPU waits for memory access

Figure 2: Single Bus



Multi Bus Architecture

Separate buses are used for:

- Memory
- CPU
- I/O

Multiple transfers occur simultaneously.

Advantages:

- Higher speed
- Parallel communication
- Reduced congestion
- Better performance

Figure 3: Multi Bus

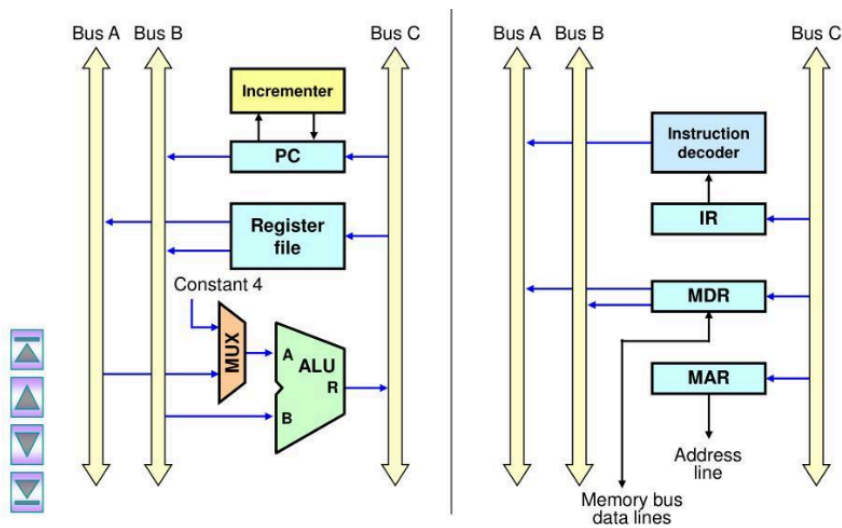
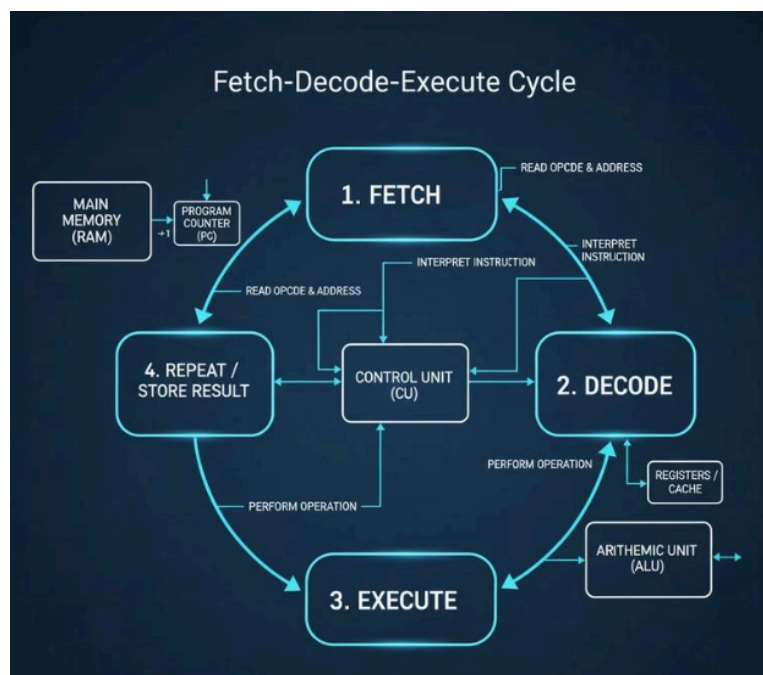


Table 2: Single Bus vs Multi Bus

Feature	Single Bus	Multi Bus
Number of Buses	One	Multiple
Speed	Low	High
Bus Contention	High	Low
Data Transfer	Sequential	Parallel
Performance	Moderate	Excellent

4. Instruction Execution Cycle



Improvements

Cache Memory

Stores frequently used data.

Benefits:

- Faster access
- Less RAM usage
- Reduced execution time

Instruction Pipelining

Executes multiple instructions simultaneously.

Example:

Cycle	Instruction
1	Fetch I1
2	Decode I1 + Fetch I2
3	Execute I1 + Decode I2 + Fetch I3

DMA (Direct Memory Access)

Transfers data directly from I/O to memory without CPU involvement.

Advantages:

- Faster transfers
- CPU remains free for processing

Branch Prediction

Predicts the next instruction before execution.

Benefits:

- Fewer pipeline stalls
- Better CPU utilization

Multi-Core Processor

Each core processes different camera feeds.

Benefits:

- Parallel execution
- Faster threat detection

Table 3: Performance Improvement Techniques

Technique	Benefit
Cache Memory	Faster memory access
DMA	Reduces CPU workload
Pipelining	Executes multiple instructions
Branch Prediction	Reduces delays
Multi-Core CPU	Parallel processing

5. Problem-Solving Approach

To solve the lag issue:

- 1. Replace the single bus with a multi-bus architecture.
- 2. Increase RAM capacity.
- 3. Use cache memory.
- 4. Apply instruction pipelining.
- 5. Use DMA for video transfer.
- 6. Install a multi-core processor.
- 7. Optimize AI algorithms.

These steps reduce waiting time and improve system performance.

6. Python Simulation Code

```
import random

print("SMART CITY SURVEILLANCE SYSTEM")
print("-" * 40)

frames = 5

for i in range(1, frames + 1):

    cpu = random.randint(50,100)

    print("\nFrame:", i)
    print("CPU Usage:", cpu,"%")

    if cpu > 85:
        print("Status : High Traffic")
        print("Threat Detection : DELAYED")
    else:
        print("Status : Normal")
        print("Threat Detection : SUCCESS")
```

Output

SMART CITY SURVEILLANCE SYSTEM

Frame: 1
CPU Usage: 62 %
Status : Normal
Threat Detection : SUCCESS

Frame: 2
CPU Usage: 94 %
Status : High Traffic
Threat Detection : DELAYED

Frame: 3
CPU Usage: 75 %
Status : Normal
Threat Detection : SUCCESS

Frame: 4
CPU Usage: 98 %
Status : High Traffic
Threat Detection : DELAYED

Frame: 5
CPU Usage: 69 %
Status : Normal
Threat Detection : SUCCESS

7. Accuracy of Solution

The proposed solution uses a **multi-bus architecture, cache memory, DMA, instruction pipelining, and multi-core processors** to reduce CPU waiting time and bus congestion. These improvements increase throughput, lower execution time, and enable real-time threat detection even during peak traffic conditions.

8. Conclusion

The performance of a smart city surveillance system depends on the efficient interaction between the CPU, memory, and I/O devices. Heavy video traffic during peak hours causes CPU overload, memory bottlenecks, and bus congestion, leading to delayed threat detection. By replacing a single-bus architecture with a multi-bus architecture and implementing techniques such as cache memory, DMA, instruction pipelining, and multi-core processing, the system can achieve faster data transfer, reduced latency, and reliable real-time surveillance.

Question 2

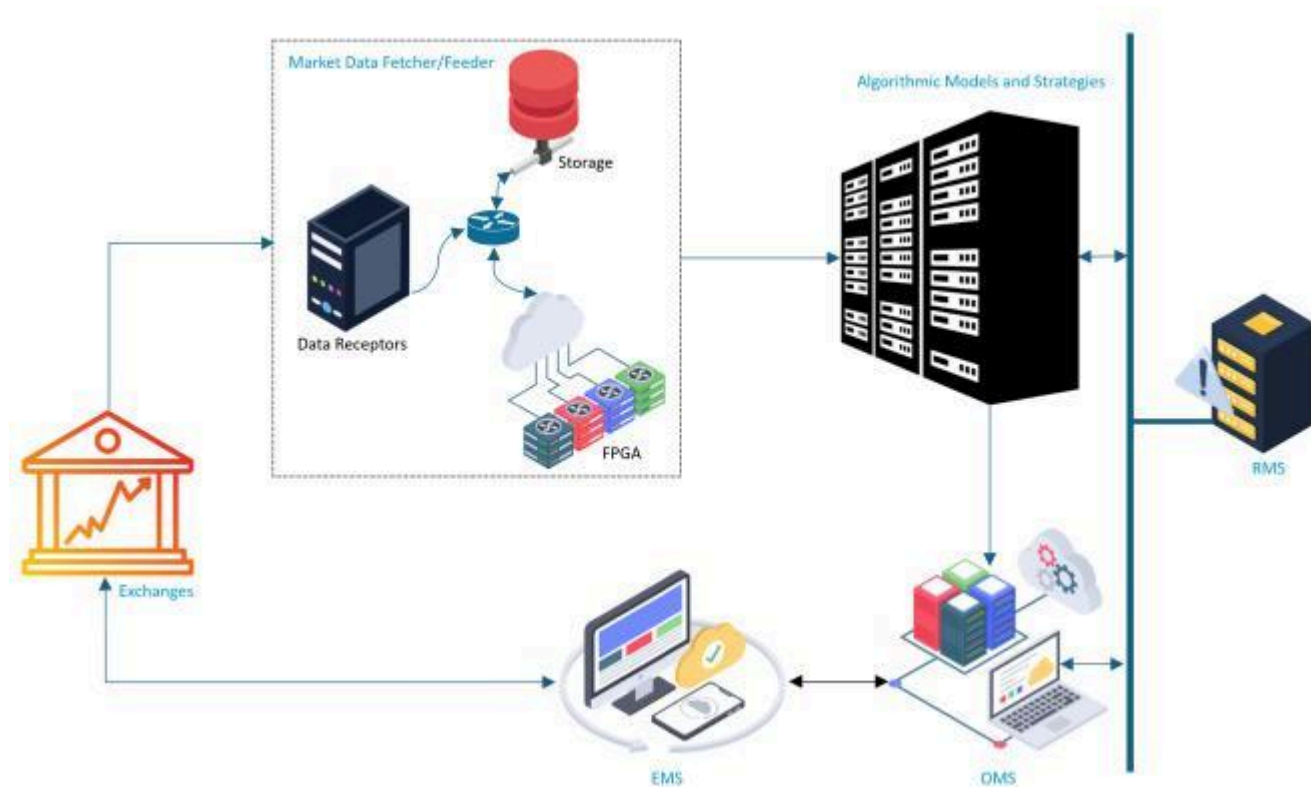
Problem Statement

A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

1. Understanding the Problem Context

A real-time stock trading system processes thousands of buy and sell orders every second. The processor executes millions of instructions to validate transactions, update prices, and communicate with stock exchanges. During periods of high market activity, the CPU experiences delays because of frequent memory access and branch instructions, which increase the **Cycles Per Instruction (CPI)**. A high CPI means each instruction takes more clock cycles to execute, reducing the system's overall performance and increasing transaction latency.

Figure 1: Stock Trading System Architecture



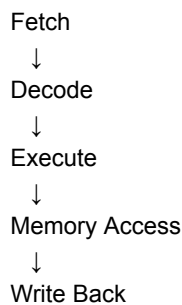
Caption:

Figure 1: Data flow in a real-time stock trading system.

2. Application of Concepts

Fetch–Decode–Execute Cycle

The processor executes every instruction through the following stages:



Fetch

- The CPU fetches instructions from memory.
- Frequent memory access increases fetch time.

Decode

- The Control Unit decodes the instruction.
- Complex instructions require additional cycles.

Execute

- The Arithmetic Logic Unit (ALU) performs calculations.
- Branch instructions may interrupt execution.

Memory Access

- Data is read from or written to RAM.
- Cache misses increase execution time.

Write Back

- The final result is stored in registers or memory.

What is CPI?

CPI (Cycles Per Instruction) represents the average number of clock cycles required to execute one instruction.

Formula

$$\text{CPI} = \text{Total Clock Cycles} / \text{Total Instructions Executed}$$

Higher CPI results in lower CPU performance.

Factors Increasing CPI

- Frequent memory access
- Cache misses
- Branch instructions
- Pipeline stalls
- CPU waiting for I/O operations

Table 1: Factors Affecting CPI

Factor	Effect on Performance
Cache Miss	Increases memory access time
Branch Instructions	Causes pipeline stalls
High Memory Access	Increases execution time
Slow I/O	Delays CPU execution
Pipeline Hazards	Reduces throughput

3. Problem Analysis

During high stock market activity:

- Millions of transactions arrive simultaneously.
- CPU repeatedly fetches instructions from memory.
- Cache misses force access to slower RAM.
- Branch instructions interrupt the instruction pipeline.
- The CPU waits for memory responses.
- CPI increases.
- Transaction processing becomes slower.

4. Methods to Reduce CPI

1. Cache Memory

Cache stores frequently used instructions and data.

Advantages

- Faster memory access
- Reduced memory latency
- Lower CPI

2. Instruction Pipelining

Multiple instructions execute simultaneously.

Example:

Clock Cycle	Operations
1	Fetch I1
2	Decode I1 + Fetch I2
3	Execute I1 + Decode I2 + Fetch I3

Benefits

- Higher throughput

- Lower execution time

3. Branch Prediction

The processor predicts branch outcomes before execution.

Benefits

- Fewer pipeline stalls
- Faster execution

4. Out-of-Order Execution

Independent instructions execute before waiting instructions.

Benefits

- Better CPU utilization
- Reduced idle time

5. Multi-Core Processor

Different cores process different stock transactions.

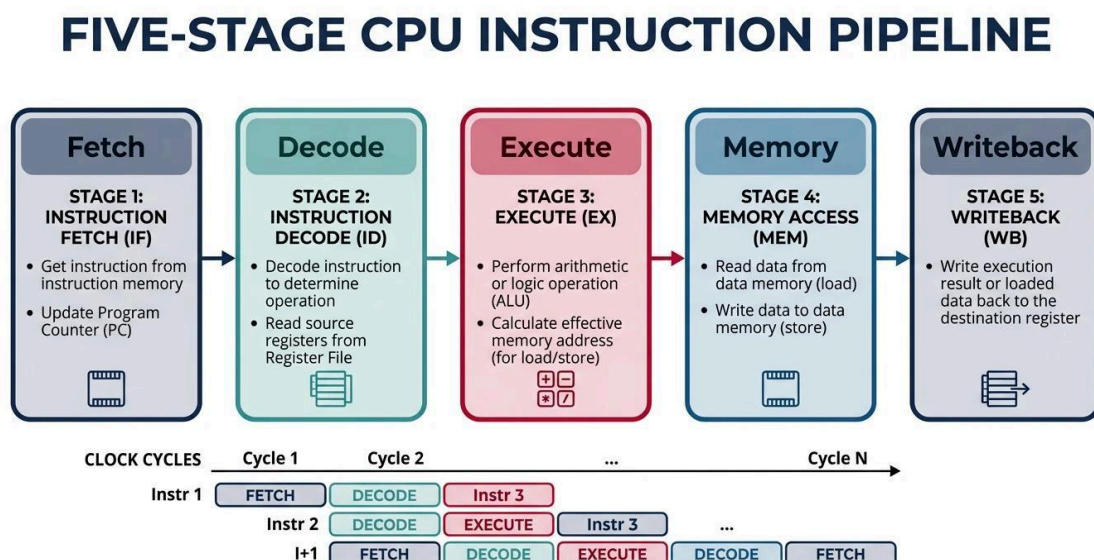
Benefits

- Parallel processing
- Increased throughput

6. High-Speed Memory

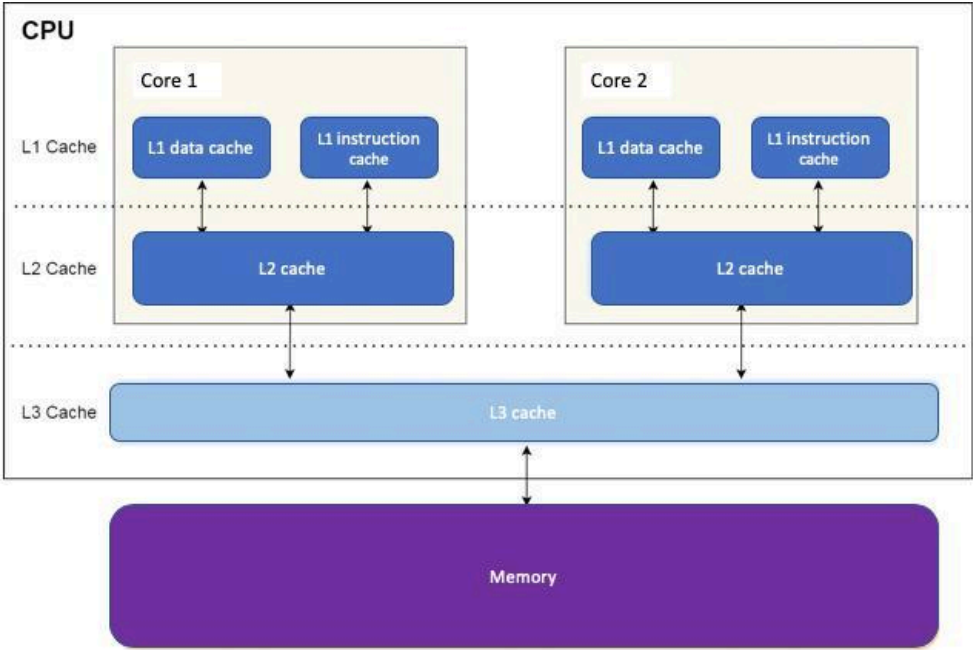
Using DDR5 RAM or faster cache memory reduces memory access time.

Figure 2: Instruction Pipeline



Caption:Figure 2: CPU instruction pipeline.

Figure 3: Cache Memory Architecture



Caption:
Figure 3: Cache memory reduces memory access time.

Table 2: Performance Improvement Techniques

Technique	Benefit
Cache Memory	Faster memory access
Branch Prediction	Reduces pipeline stalls
Pipelining	Executes multiple instructions simultaneously
Multi-Core CPU	Parallel execution
High-Speed RAM	Reduces memory latency
Out-of-Order Execution	Improves CPU utilization

5. Problem-Solving Approach

To reduce execution time in the stock trading application:

1. Increase cache memory size.
2. Implement instruction pipelining.
3. Use branch prediction.

4. Upgrade to multi-core processors.
5. Use faster memory modules.
6. Reduce unnecessary branch instructions.
7. Optimize algorithms for fewer memory accesses.

These improvements reduce CPI and improve transaction processing speed.

6. Python Simulation Code

```
import random

print("REAL-TIME STOCK TRADING SYSTEM")
print("-" * 40)

transactions = 5

for i in range(1, transactions + 1):

    cpi = round(random.uniform(0.8, 3.0), 2)

    print("\nTransaction", i)
    print("CPI :", cpi)

    if cpi > 2:
        print("Status : HIGH CPI")
        print("Processing : DELAYED")
    else:
        print("Status : NORMAL CPI")
        print("Processing : SUCCESSFUL")
```

Output

REAL-TIME STOCK TRADING SYSTEM

Transaction 1
CPI : 1.25
Status : NORMAL CPI
Processing : SUCCESSFUL

Transaction 2
CPI : 2.64
Status : HIGH CPI
Processing : DELAYED

Transaction 3
CPI : 1.78
Status : NORMAL CPI
Processing : SUCCESSFUL

Transaction 4
CPI : 2.91
Status : HIGH CPI
Processing : DELAYED

Transaction 5
CPI : 1.12

7. Accuracy of Solution

The proposed solution improves CPU performance by reducing **Cycles Per Instruction (CPI)**. Techniques such as **cache memory, instruction pipelining, branch prediction, multi-core processors, high-speed memory**, and **out-of-order execution** reduce execution time, minimize processor stalls, and increase throughput. These enhancements enable the stock trading system to process transactions efficiently even during periods of heavy trading activity.

8. Conclusion

In a real-time stock trading application, high CPI caused by frequent memory access and branch instructions leads to slower transaction processing. Optimizing the fetch–decode–execute cycle through architectural improvements such as cache memory, instruction pipelining, branch prediction, faster memory, and multi-core processors significantly improves CPU performance. These enhancements reduce execution delays, increase throughput, and ensure reliable real-time transaction processing.

Question 3

Problem Statement

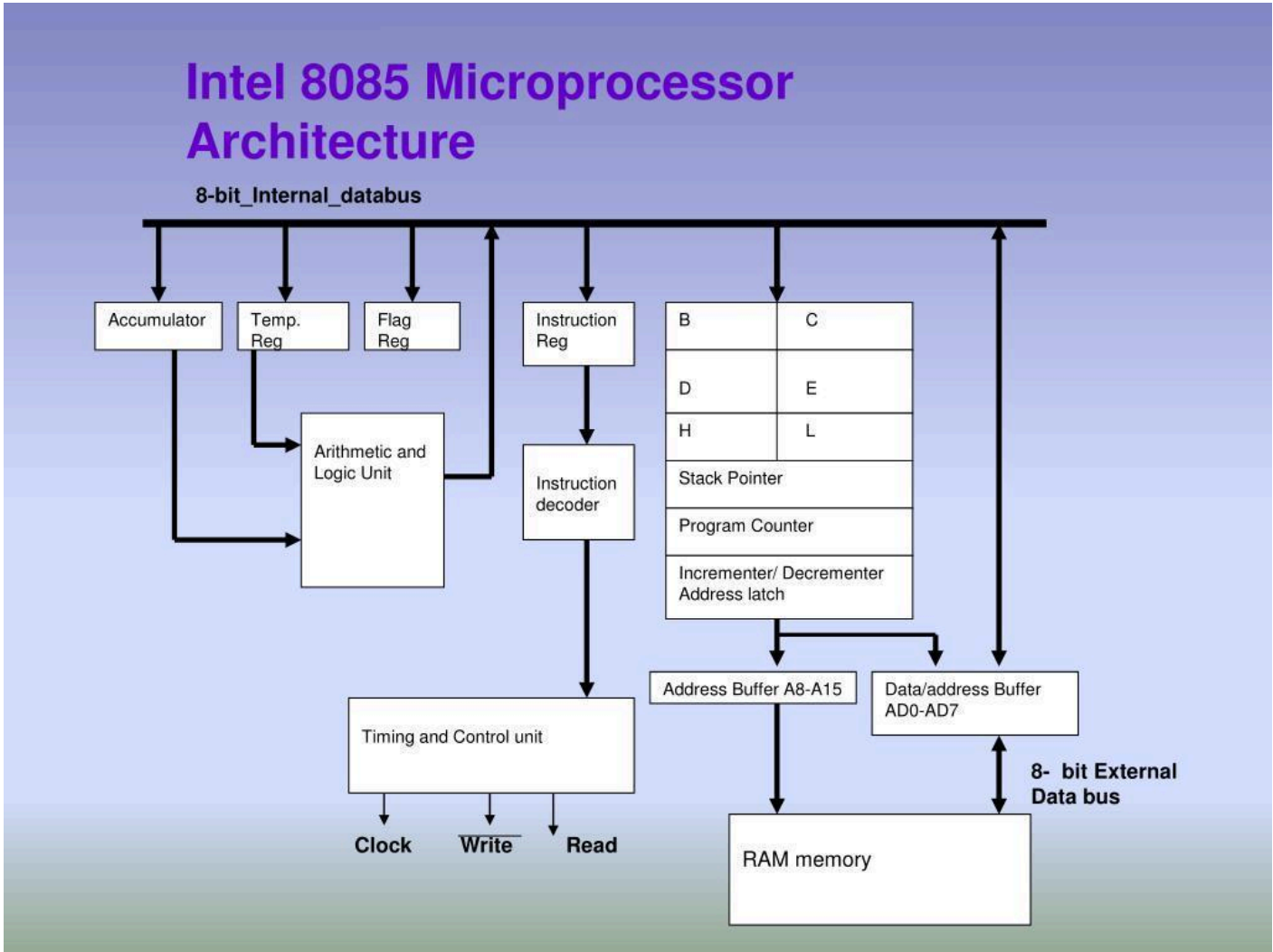
An embedded system based on an **8085 microprocessor** is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of the 8085 can be effectively used to ensure accurate and reliable operation.

1. Understanding the Problem Context

An industrial temperature control system continuously monitors temperature using sensors connected to an **8085 microprocessor**. The sensor readings are processed by the CPU, and based on the measured temperature, the microprocessor controls actuators such as fans, heaters, or alarms.

Sometimes, incorrect temperature values are processed because the program uses **improper addressing modes**. As a result, the CPU may read data from the wrong memory location or register, leading to incorrect decisions and unreliable operation.

Figure 1: 8085-Based Temperature Control System



Caption:
Figure 1: Temperature control system using the 8085 microprocessor.

2. Application of Concepts

8085 Microprocessor

The **8085** is an **8-bit microprocessor** with:

- Arithmetic Logic Unit (ALU)
- Control Unit (CU)
- Accumulator
- Registers (A, B, C, D, E, H, L)
- Program Counter (PC)
- Stack Pointer (SP)

It processes sensor data using different addressing modes.

Addressing Modes in 8085

1. Immediate Addressing Mode

The data is directly included in the instruction.

Example:

`MVI A,25H`

Meaning:

Load **25H** directly into Accumulator **A**.

2. Register Addressing Mode

Data is stored in another register.

Example:

`MOV A,B`

Meaning:

Copy the contents of Register **B** into Register **A**.

3. Direct Addressing Mode

The instruction contains the memory address.

Example:

`LDA 2050H`

Meaning:

Load data from memory location **2050H** into the Accumulator.

4. Register Indirect Addressing Mode

The memory address is stored in a register pair (HL).

Example:

MOV A,M

Meaning:

Load the data pointed to by the **HL** register pair into the Accumulator.

Table 1: Addressing Modes

Addressing Mode	Example	Application
Immediate	MVI A,25H	Load constant value
Register	MOV A,B	Transfer between registers
Direct	LDA 2050H	Read sensor data from memory
Register Indirect	MOV A,M	Access memory through HL

3. Influence of Addressing Modes on Data Access

Each addressing mode determines **how the CPU accesses data**.

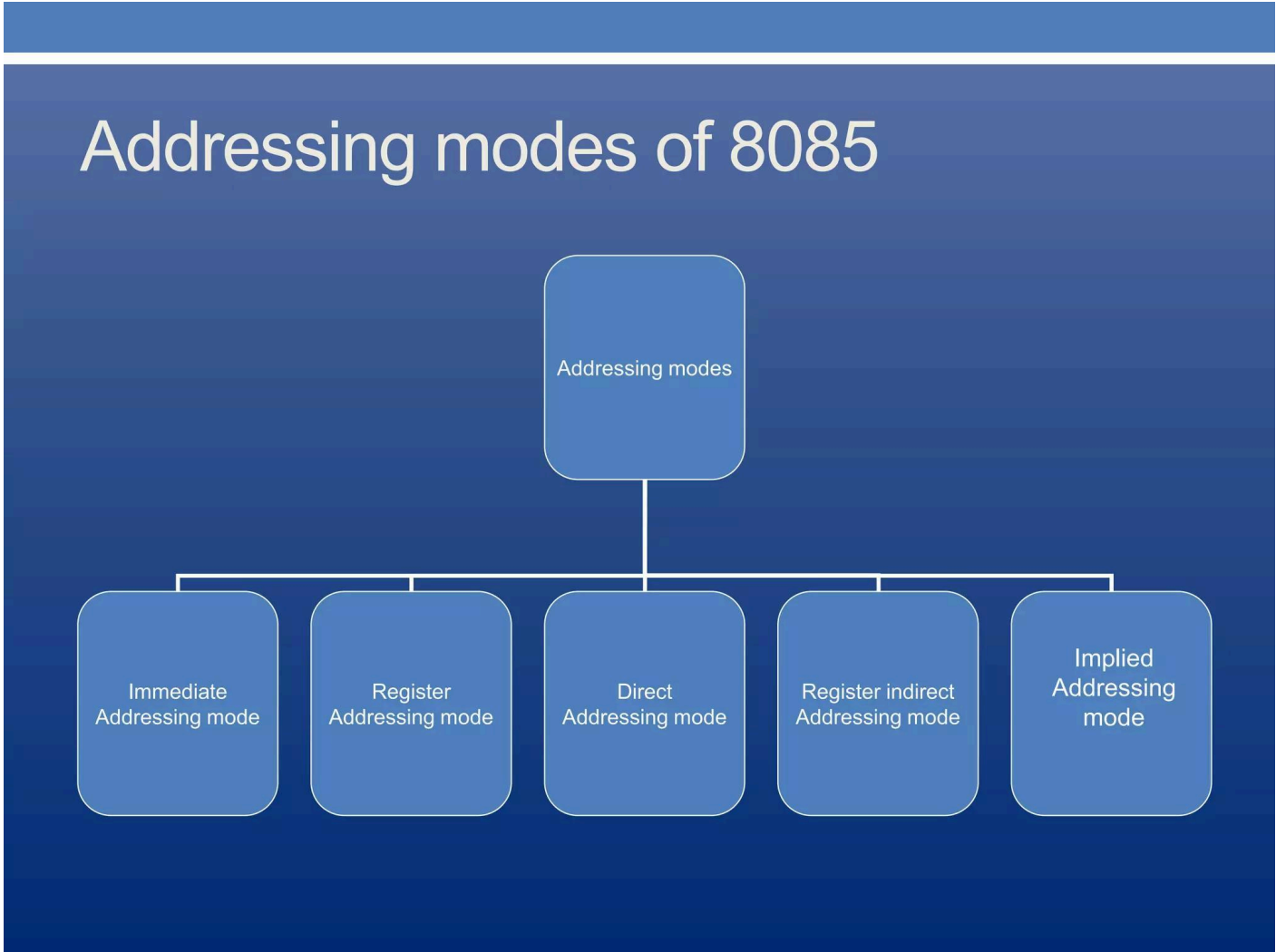
- **Immediate Mode** is fast because the data is available within the instruction.
- **Register Mode** provides faster execution since data is stored inside CPU registers.
- **Direct Mode** accesses a fixed memory location.
- **Register Indirect Mode** accesses data using the address stored in the HL register pair, providing flexibility.

If the wrong addressing mode is selected, the CPU may access incorrect data, resulting in inaccurate temperature readings.

4. Possible Causes of Incorrect Temperature Readings

- Wrong memory address used in the program.
- Incorrect HL register value.
- Sensor data stored in the wrong memory location.
- Incorrect instruction selection.
- Register values overwritten during execution.
- Programming errors in data transfer.
- Faulty sensor or memory corruption.

Figure 2: Addressing Modes in 8085



Caption:
Figure 2: Different addressing modes used by the 8085 microprocessor.

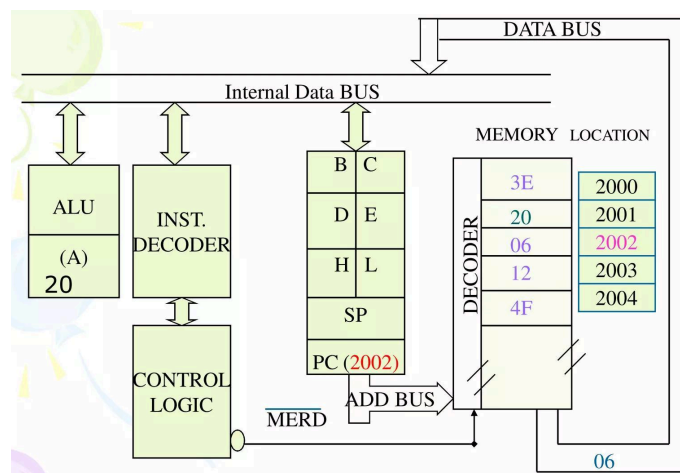
5. Effective Use of the 8085 Instruction Set

To improve system reliability, the following instructions can be used correctly.

Instruction	Function
MVI	Load immediate data
MOV	Transfer data between registers
LDA	Load Accumulator from memory
STA	Store Accumulator into memory

IN	Read sensor data
OUT	Send control signal to actuator
CMP	Compare temperature values
JZ	Jump if temperature is equal
JC	Jump if Carry flag is set
HLT	Stop program execution

Figure 3: Instruction Execution Flow



Caption:
Figure 3: Instruction execution cycle of the 8085 microprocessor.

6. Problem-Solving Approach

To ensure accurate temperature measurement:

1. Use the correct addressing mode for each instruction.
2. Store sensor readings in the correct memory location.
3. Validate register values before processing.
4. Use **LDA** and **STA** carefully for memory access.
5. Compare readings using **CMP**.
6. Use conditional jump instructions (**JZ**, **JC**) for decision making.
7. Test the program thoroughly before deployment.

These practices improve accuracy and system reliability.

7. Python Simulation Code

```
import random

print("INDUSTRIAL TEMPERATURE CONTROL SYSTEM")

print("-" * 40)

for i in range(1, 6):

    temperature = random.randint(20, 100)

    print("\nSensor Reading:", temperature, "°C")

    if temperature > 70:

        print("Action : Cooling Fan ON")

    else:

        print("Action : Temperature Normal")
```

Output

INDUSTRIAL TEMPERATURE CONTROL SYSTEM

Sensor Reading: 35 °C

Action : Temperature Normal

Sensor Reading: 82 °C

Action : Cooling Fan ON

Sensor Reading: 55 °C

Action : Temperature Normal

Sensor Reading: 91 °C

Action : Cooling Fan ON

Sensor Reading: 41 °C

Action : Temperature Normal

8. Accuracy of Solution

Using the correct **8085 addressing modes** ensures that the CPU accesses the intended memory locations and registers, preventing incorrect temperature readings. Proper use of instructions such as **MVI**, **MOV**, **LDA**, **STA**, **CMP**, **IN**, and **OUT** enables reliable sensor data processing and accurate actuator control. These practices improve the overall efficiency, accuracy, and stability of the embedded temperature control system.

9. Conclusion

The addressing modes of the **8085 microprocessor** play an important role in determining how data is accessed and processed. Incorrect use of addressing modes can lead to wrong sensor readings, improper actuator operation, and unreliable system performance. By selecting the correct addressing mode, using appropriate instructions, and validating memory and register values, the industrial temperature control system can operate accurately, efficiently, and reliably.